Skill Week-10

Session-19

Task 1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```
kiran@k:~/OSSP$ cd ~/OSSP
kiran@k:~/OSSP$ nano 19_task1.c
kiran@k:~/OSSP$ gcc 19_task1.c -o signal_handler -Wall -Wextra
kiran@k:~/OSSP$ ./signal_handler
Process started. PID = 1685
Press Ctrl+C to send SIGINT.
Process is running...
Process is running...
Process is running...
^C
Signal received: 2
Signal handler executed successfully.
Process is running...
Process is running...
Process is running...
^C
Signal received: 2
Signal handler executed successfully.
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_received = 0;

void signal_handler(int sig, siginfo_t *info, void *context)
{
    // Prevent unused parameter warning
    (void)context;

    signal_received = sig;

    printf("\nSignal received!\n");
    printf("Signal number: %d\n", sig);
    printf("Sender PID: %d\n", info->si_pid);

    if (sig == SIGINT)
    {
        printf("SIGINT received. Program will continue running.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("SIGTERM received. Program will terminate.\n");
        exit(0);
    }
}

int main()
{
    struct sigaction action;

    // Clear the structure
    memset(&action, 0, sizeof(action));

    // Set signal handler
    action.sa_sigaction = signal_handler;
    action.sa_flags = SA_SIGINFO;

    // Initialize signal mask
    sigemptyset(&action.sa_mask);

    // Register SIGINT handler
    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }

    // Register SIGTERM handler
```

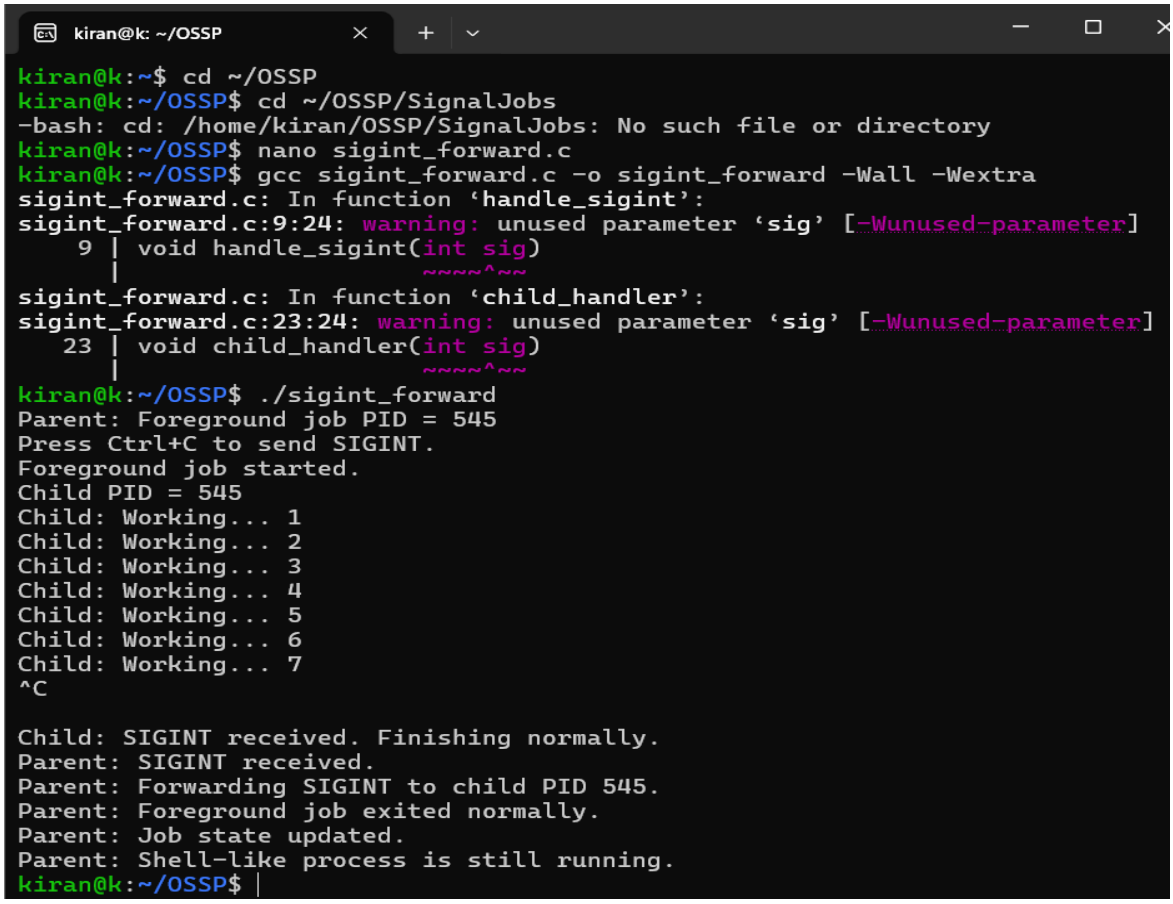
```
    // Register SIGTERM handler
    if (sigaction(SIGTERM, &action, NULL) == -1)
    {
        perror("sigaction SIGTERM");
        return 1;
    }

    printf("====================================\n");
    printf("      SIGNAL HANDLER PROGRAM\n");
    printf("====================================\n");
    printf("Program started successfully.\n");
    printf("PID = %d\n", getpid());
    printf("\nPress Ctrl+C to send SIGINT.\n");
    printf("Use 'kill -TERM PID' to send SIGTERM.\n");
    printf("====================================\n");

    // Keep program running
    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }

    return 0;
}
```

Task 2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void sigint_handler(int sig)
{
    // Prevent unused parameter warning
    (void)sig;

    printf("\nParent: SIGINT received.\n");

    if (foreground_pid > 0)
    {
        printf("Parent: Forwarding SIGINT to child PID %d.\n",
               foreground_pid);

        kill(foreground_pid, SIGINT);
    }
}

int main()
{
    struct sigaction action;
    pid_t pid;
    int status;

    // Configure SIGINT handler
    action.sa_handler = sigint_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGINT, &action, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
```

```
    {
        // Child process
        printf("Foreground job started.\n");
        printf("Child PID = %d\n", getpid());

        for (int i = 1; i <= 20; i++)
        {
            printf("Child: Working... %d\n", i);
            sleep(1);
        }

        printf("Child: Finished normally.\n");
        exit(0);
    }

    // Parent process
    foreground_pid = pid;

    printf("Parent: Foreground job PID = %d\n", pid);
    printf("Press Ctrl+C to send SIGINT.\n");

    // Wait for child
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Parent: Foreground job exited normally.\n");
    }
    else if (WIFSIGNALED(status))
    {
        printf("Parent: Foreground job terminated by signal %d.\n",
               WTERMSIG(status));
    }

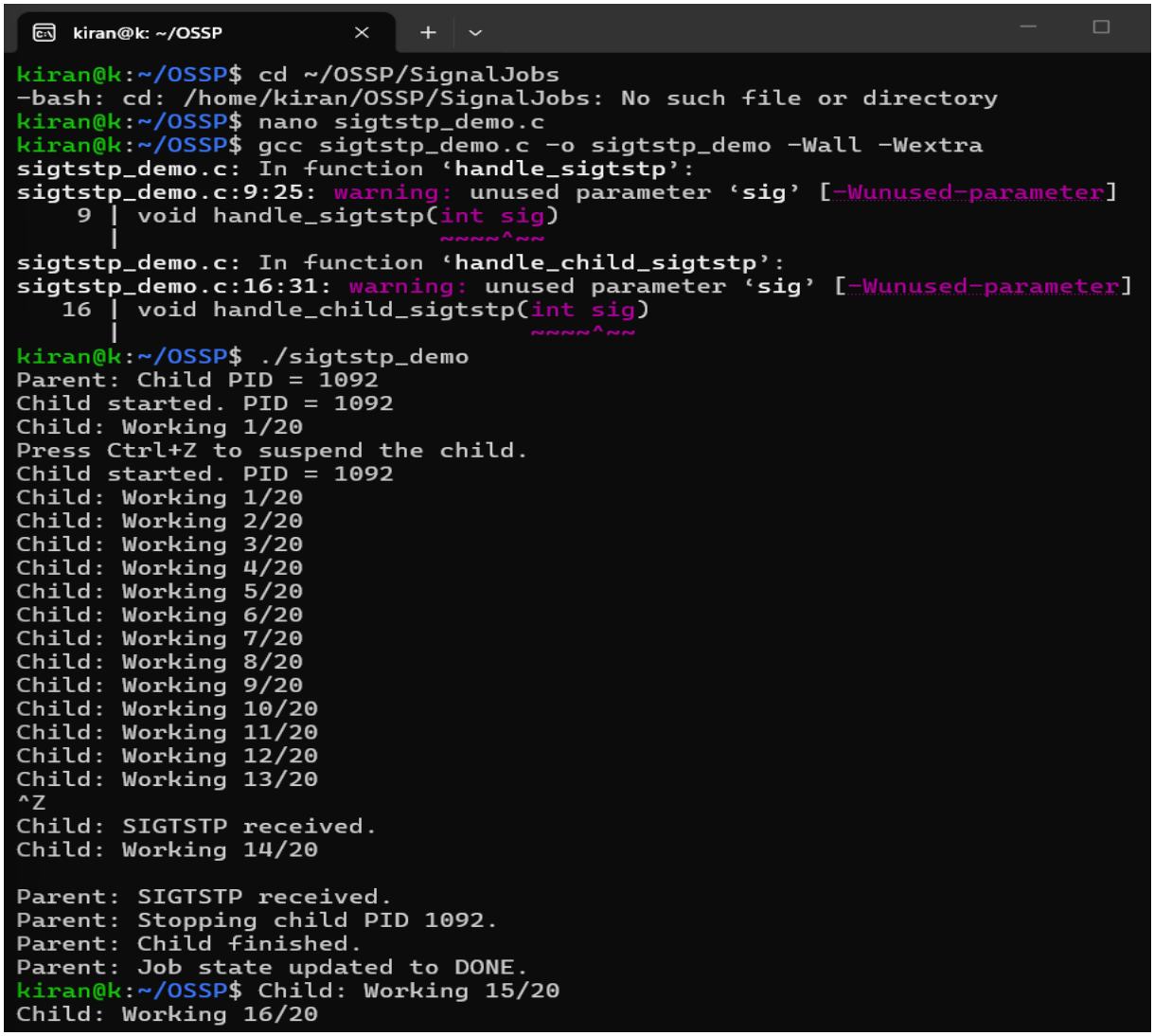
    foreground_pid = -1;

    printf("Parent: Job state updated.\n");
    printf("Parent: Shell-like process is still running.\n");

    return 0;
}
```

Session-20

Task 1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid = -1;

void sigtstp_handler(int sig)
{
    // Prevent unused parameter warning
    (void)sig;

    printf("\nParent: SIGTSTP received.\n");

    if (child_pid > 0)
    {
        printf("Parent: Stopping child PID %d.\n", child_pid);

        kill(child_pid, SIGSTOP);
    }
}

int main()
{
    struct sigaction action;
    pid_t pid;
    int status;

    // Configure SIGTSTP handler
    action.sa_handler = sigtstp_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    if (sigaction(SIGTSTP, &action, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child started. PID = %d\n", getpid());
```

```
        // Child process
        printf("Child started. PID = %d\n", getpid());

        for (int i = 1; i <= 20; i++)
        {
            printf("Child: Working %d/20\n", i);
            sleep(1);
        }

        printf("Child: Finished.\n");
        exit(0);
    }

    // Parent process
    child_pid = pid;

    printf("Parent: Child PID = %d\n", child_pid);
    printf("Press Ctrl+Z to suspend the child.\n");

    // Wait until child is stopped
    waitpid(child_pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        printf("\nParent: Child is now STOPPED.\n");
        printf("Parent: Job state updated to STOPPED.\n");

        printf("Parent: Sending SIGCONT...\n");

        // Resume child
        kill(child_pid, SIGCONT);

        printf("Parent: Child resumed.\n");
    }

    // Wait until child finishes
    waitpid(child_pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Parent: Child finished.\n");
        printf("Parent: Job state updated to DONE.\n");
    }

    return 0;
}
```

Task 2: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
kiran@k:~/OSSP$ cd ~/OSSP
kiran@k:~/OSSP$ nano process_group.c
kiran@k:~/OSSP$ gcc process_group.c -o process_group -Wall -Wextra
kiran@k:~/OSSP$ ./process_group
Child 1: PID = 1158
Child 1: PGID = 1158
Child 1: Working 1/5

Parent PID = 1157
Child 2: PID = 1159
Process Group ID = 1158
Child 2: PGID = 1158
Transferring terminal control to process group 1158...
Child 2: Working 1/5
Foreground process group is now 1158
Child 1: Working 2/5
Child 2: Working 2/5
Child 1: Working 3/5
Child 2: Working 3/5
Child 1: Working 4/5
Child 2: Working 4/5
Child 1: Working 5/5
Child 2: Working 5/5

Restoring terminal control to parent...

[1]+  Stopped                 ./process_group
kiran@k:~/OSSP$
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    pid_t child1;
    pid_t child2;
    pid_t pgid;
    int status;

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        // First child becomes process-group leader
        setpgid(0, 0);

        printf("Child 1: PID = %d\n", getpid());
        printf("Child 1: PGID = %d\n", getpgrp());

        for (int i = 1; i <= 5; i++)
        {
            printf("Child 1: Working %d/5\n", i);
            sleep(1);
        }

        exit(0);
    }

    // Parent ensures child 1 is group leader
    setpgid(child1, child1);

    pgid = child1;

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork");
        return 1;
    }
```

```
    if (child2 == 0)
    {
        // Put second child in child 1's group
        setpgid(0, pgid);

        printf("Child 2: PID = %d\n", getpid());
        printf("Child 2: PGID = %d\n", getpgrp());

        for (int i = 1; i <= 5; i++)
        {
            printf("Child 2: Working %d/5\n", i);
            sleep(1);
        }

        exit(0);
    }

    // Parent places child 2 into the same group
    setpgid(child2, pgid);

    printf("\nParent PID = %d\n", getpid());
    printf("Process Group ID = %d\n", pgid);

    printf("Transferring terminal control to process group %d...\n",
           pgid);

    if (tcsetpgrp(STDIN_FILENO, pgid) == -1)
    {
        perror("tcsetpgrp");
    }

    printf("Foreground process group is now %d\n", pgid);

    // Wait for both children
    waitpid(child1, &status, 0);
    waitpid(child2, &status, 0);

    // Restore terminal control
    printf("\nRestoring terminal control to parent...\n");

    if (tcsetpgrp(STDIN_FILENO, getpgrp()) == -1)
    {
        perror("tcsetpgrp");
    }

    printf("Terminal control restored.\n");
    printf("All child processes completed.\n");

    return 0;
}
```